Adaboost

1) Weak learners : accuracy little bit greater than 50%.

- Adaboost combines weak learners and gives good accuray.

2) decision stumps max_depth=1

- decision tree 1 with high information gain

3) +1 and -1 not zero '0'
 yes no.

Adaboost is stagewise additive method.

Steps:

- One ~~...~~ method works. also weight calc
- then it tells another method [decision stump] don't do this mistake.

:

tell to another decision stumps & improves.

and at last ~~total~~ ~~weight~~ net position is calculated by weight calculated.

weight

$h(x) = \text{sign}(\alpha_1 \times h1(x) + \alpha_2 \cdot h2(x) + \alpha_3 \times h3(x))$

calc sign either 1 or -1

Page No.
Date : / /

Adaboost- step by step.

- Let assume 2 col^n x_1, x_2, y.
& 5 rows.

1: Assign equal weight to every rows.
$\frac{1}{\text{no. of rows}} \rightarrow \frac{1}{5} \rightarrow 0.20$

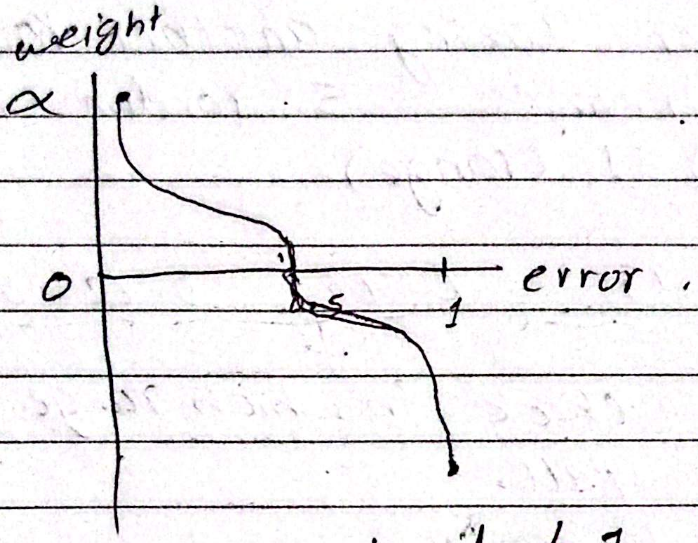
2: decision stump -1.
3: cal y-pred.
4: cal α_1 [weight].
↳ depends on error rate.

Q Which model reliable?
Error rate: [1- accuracy rate]
a> 0%.
b> 100%
c> 50%

ans: a & b same, c is very dangerous.

~~0% is good~~
But doing negative of error rate 100%, it is equal to the error rat of 50%.

Page No.
Date : / /



$$\alpha = \frac{1}{2} \ln\left(\frac{1 - \text{error}}{\text{error}}\right)$$

error = Σ weight of mis predicted values.

Cal : α_1

5: model increases the feature importance of the errored row and warns another model not to repeat that mistake. [boost]
By, increasing weight of misclassified point. So; booting.

For weight updates:

For miss classified.

$$\boxed{new\_wt = current\_wt \times e^{\alpha_1}}$$

For correct classified.

$$\boxed{new\_wt = current\_wt \times e^{-\alpha_1}}$$

~~6: This loop repeats~~:

6: make ranges according to weight for each rows (0 - 0.16, 0.16 - 0.25, ...)

Page No.
Date: / /

7: For taking another row, it generates 5 random no. betn 0 - 1 (range)

0.13 0.43 0.62 0.5 0.8

check in which range does it fall.

then, choose that row

$\alpha_2 \rightarrow +1$

8: And this loop repeats till decision stumps.

9: Final pred:

Sign of

$P = \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3$.

$$\underbrace{lr \times}_{\text{learning rate}} \alpha = \frac{1}{2} \log\left(\frac{1 - error}{error}\right)$$

- If $lr \downarrow$, then amplitude decreases.
- It slow down learning, to decrease overfitting.